

LAPORAN TUGAS BESAR

STRATEGI ALGORITMA

*(Pemanfaatan Algoritma Greedy dalam pembuatan bot permainan Robocode
Tank Royale)*



Dosen Pengampu:
Winda Yulita, M.Cs.

DISUSUN OLEH:



Sena Permatasari	124140018
Wulandari Lubis	124140066
Sere Sri Rezeki Sinaga	124140150

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
2026

DAFTAR ISI

DAFTAR ISI.....	1
DAFTAR TABEL.....	3
BAB I DESKRIPSI TUGAS.....	4
1.1 Latar Belakang.....	4
BAB II LANDASAN TEORI.....	10
2.1 Algoritma Greedy.....	10
2.2 Cara Kerja Program Secara Umum.....	11
2.3 Proses Scanning oleh Bot.....	12
2.4 Evaluasi dan Pemilihan Aksi.....	12
2.5 Eksekusi dan Perulangan Proses.....	13
2.6 Implementasi Strategi Greedy.....	13
BAB III APLIKASI STRATEGI GREEDY.....	15
3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Algoritma Greedy.....	15
3.2 Eksplorasi Alternatif Solusi Greedy.....	16
3.2.1 Alternatif 1 — Distance-Based Adaptive Firepower (Bot Gacor).....	16
3.2.2 Alternatif 2 — Energy-Aware Adaptive Strategy (Bot Syududu).....	17
3.2.3 Alternatif 3 — Survival-First Evasion (Bot Kaciw).....	17
3.2.4 Alternatif 4 — Always-Aggressive Maximum Damage (Bot Kacau).....	18
3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi.....	19
3.4 Strategi Greedy yang Dipilih dan Alasan Pemilihan.....	20
BAB IV IMPLEMENTASI DAN PENGUJIAN.....	22
4.1 Implementasi Pseudocode Keempat Alternatif Solusi.....	22
4.1.1 Pseudocode Bot Gacor (Bot Utama).....	22
4.1.2 Pseudocode Bot Syududu (Alternatif 1).....	23
4.1.3 Pseudocode Bot Kaciw (Alternatif 2).....	24
4.1.4 Pseudocode Bot Kacau (Alternatif 3).....	24
4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur Bot Utama (Gacor).....	25
4.2.1 Struktur Data.....	25
4.2.2 Fungsi.....	26
4.2.3 Prosedur.....	27
4.2.4 Alur Keputusan Greedy.....	27
4.3 Pengujian (Hasil Pertandingan).....	28
4.4 Analisis Hasil Pengujian.....	29

4.4.1 Analisis Performa Kemenangan Berdasarkan Komponen Skor.....	29
4.4.2 Kapan Strategi Greedy Berhasil Mencapai Nilai Optimal?.....	30
4.4.3 Kapan Strategi Greedy Gagal Mencapai Nilai Optimal?.....	30
BAB V KESIMPULAN DAN SARAN.....	32
5.1 Kesimpulan.....	32
5.2 Saran.....	32
LAMPIRAN.....	33
Lampiran A — Tautan Repository GitHub.....	33
Lampiran B — Tautan Video YouTube.....	33
Lampiran C — Tabel Checklist Spesifikasi.....	33

DAFTAR TABEL

Tabel 1. Mapping Elemen Algoritma Greedy pada Robocode Tank Royale.....	15
Tabel 2. Aturan Seleksi Firepower Bot Gacor.....	16
Tabel 3. Aturan Seleksi Firepower Bot Syududu.....	17
Tabel 4. Aturan Seleksi Firepower Bot Kaciw.....	18
Tabel 5. Aturan Aksi Greedy Bot Kacau.....	19
Tabel 6. Perbandingan Efisiensi dan Efektivitas Keempat Alternatif Solusi.....	19
Tabel 7. Struktur Data Bot Gacor.....	25
Tabel 8. Daftar Fungsi Bot Gacor.....	26
Tabel 9. Daftar Prosedur Bot Gacor.....	27
Tabel 10. Hasil Pengujian dengan bot asisten.....	28

BAB I

DESKRIPSI TUGAS

1.1 Latar Belakang

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari [versi asli/pertama permainan ini](#). Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan **strategi greedy** dalam membuat bot ini.

Komponen-komponen dari permainan ini antara lain:

1. Rounds dan Turn

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah.

Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.

- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

Perlu diperhatikan bahwa [API \(Application Programming Interface\)](#) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri.

Pada setiap turn, bot akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai.

Perlu diketahui bahwa game engine yang akan digunakan pada tugas besar ini tidak mengikuti aturan default mengenai komponen Round & Turns.

2. Batas Waktu Giliran

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini.

Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewati turn tersebut. Jika bot melewati turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

3. Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.

- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

4. Peluru

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh.

Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

5. Panas Meriam (Gun Heat)

Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

6. Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain.

Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

7. Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:

- *Body* adalah bagian utama dari tank yang digunakan untuk menggerakkan tank.
- *Gun* digunakan untuk menembakkan peluru dan dapat berputar bersama *body* atau independen dari *body*.
- *Radar* digunakan untuk memindai posisi musuh dan dapat berputar bersama *body* atau independen dari *body*.

8. Pergerakan

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak bergantung pada kecepatan bot saat itu.

9. Berbelok

Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot.

Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar 360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran.

Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin cepat bot bergerak, semakin lambat kemampuannya untuk berbelok.

Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

10. Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar. Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar.

Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran.

Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh.

Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh.

11. Skor

Pada akhir pertempuran, setiap bot akan diranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan **poin sebesar *damage*** yang dibuat kepada bot musuh menggunakan peluru.
- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan **poin sebesar 20% dari *damage*** yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lain yang masih bertahan pada ronde tersebut mendapatkan **50 poin**.
- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan **10 poin** dikali dengan banyaknya musuh.

- **Ram Damage:** Bot mendapatkan **poin sebesar 2 kalinya *damage*** yang dibuat kepada bot musuh dengan cara menabrak.
- **Ram Damage Bonus:** Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan **poin sebesar 30% dari *damage*** yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perangkingan akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi.

BAB II LANDASAN TEORI

2.1 Algoritma Greedy

Algoritma greedy merupakan salah satu metode dalam strategi algoritma yang digunakan untuk menyelesaikan permasalahan optimasi dengan cara memilih keputusan terbaik pada setiap langkah berdasarkan kondisi saat itu (local optimum). Pendekatan ini dilakukan dengan harapan bahwa serangkaian keputusan lokal terbaik yang dipilih secara bertahap dapat menghasilkan solusi terbaik secara keseluruhan (global optimum). Prinsip utama dari algoritma greedy adalah mengambil pilihan yang dianggap paling menguntungkan pada saat tertentu tanpa mempertimbangkan kembali keputusan yang telah diambil sebelumnya.

Algoritma greedy bekerja secara bertahap dalam membangun solusi. Pada setiap langkah, algoritma akan memilih kandidat terbaik berdasarkan suatu aturan atau heuristic tertentu. Setelah keputusan dipilih, keputusan tersebut akan langsung

menjadi bagian dari solusi dan tidak akan dievaluasi ulang. Oleh karena itu, algoritma greedy tidak menggunakan proses backtracking maupun pencarian terhadap seluruh kemungkinan solusi seperti pada metode brute force.

Secara umum, algoritma greedy memiliki beberapa komponen utama, yaitu himpunan kandidat (candidate set), himpunan solusi (solution set), fungsi seleksi (selection function), fungsi kelayakan (feasibility function), dan fungsi objektif (objective function). Himpunan kandidat berisi seluruh pilihan yang dapat digunakan dalam pembentukan solusi, sedangkan himpunan solusi merupakan kumpulan kandidat yang telah dipilih menjadi solusi akhir. Fungsi seleksi digunakan untuk menentukan kandidat terbaik pada setiap langkah, fungsi kelayakan digunakan untuk memeriksa apakah kandidat memenuhi syarat permasalahan, dan fungsi objektif digunakan untuk menentukan kualitas solusi yang dihasilkan.

Karakteristik utama dari algoritma greedy adalah pengambilan keputusan yang cepat dan sederhana karena hanya berfokus pada kondisi lokal saat itu. Metode ini banyak digunakan pada berbagai permasalahan optimasi, seperti penjadwalan, pencarian jalur terpendek, pengalokasian sumber daya, serta berbagai permasalahan lain yang membutuhkan efisiensi waktu komputasi. Namun, algoritma greedy tidak selalu menghasilkan solusi global optimum karena keputusan yang dipilih hanya berdasarkan kondisi terbaik pada saat tertentu tanpa mempertimbangkan konsekuensi jangka panjang.

Meskipun demikian, algoritma greedy tetap menjadi salah satu pendekatan yang banyak digunakan karena memiliki efisiensi yang baik dan mudah diimplementasikan. Pada beberapa kasus tertentu, algoritma greedy mampu menghasilkan solusi optimal dengan waktu eksekusi yang lebih cepat dibandingkan metode pencarian menyeluruh (HARIANJA, 2019).

2.2 Cara Kerja Program Secara Umum

Program yang dikembangkan pada tugas besar ini berupa sebuah bot pada permainan Robocode Tank Royale yang bekerja secara otomatis menggunakan strategi algoritma greedy. Bot dirancang untuk menerima informasi kondisi arena permainan,

melakukan pemindaian terhadap musuh, kemudian menentukan aksi terbaik berdasarkan kondisi yang sedang terjadi pada setiap turn permainan.

Secara umum, proses kerja program dimulai ketika bot menerima data dari game engine mengenai kondisi arena, seperti posisi bot, posisi musuh yang terdeteksi radar, arah gerakan, energi, serta kondisi lingkungan di sekitar arena. Informasi tersebut kemudian diproses sebagai dasar dalam pengambilan keputusan yang akan dilakukan oleh bot.

Bot secara berkala melakukan proses scanning menggunakan radar untuk mendeteksi keberadaan musuh di dalam arena permainan. Hasil scanning digunakan untuk memperoleh informasi penting, seperti jarak musuh, arah gerakan lawan, dan posisi relatif terhadap bot. Informasi tersebut kemudian dievaluasi menggunakan pendekatan greedy untuk menentukan tindakan terbaik yang dapat dilakukan pada saat itu.

Setelah proses evaluasi dilakukan, bot akan memilih aksi dengan prioritas tertinggi berdasarkan heuristic yang digunakan. Tindakan tersebut dapat berupa menyerang musuh, bergerak menghindari serangan, mengatur arah radar, ataupun mengubah posisi agar memperoleh keuntungan dalam permainan. Seluruh keputusan diambil berdasarkan kondisi lokal saat itu tanpa melakukan perencanaan jangka panjang maupun evaluasi ulang terhadap keputusan sebelumnya.

2.3 Proses Scanning oleh Bot

Bot pada permainan Robocode Tank Royale secara berkala melakukan proses *scanning* menggunakan radar untuk mendeteksi keberadaan musuh di dalam arena permainan. Radar berfungsi sebagai alat utama bagi bot untuk memperoleh informasi mengenai kondisi lingkungan sekitar. Informasi yang diperoleh dari hasil *scanning* menjadi dasar dalam proses pengambilan keputusan pada setiap *turn* permainan.

Pada proses *scanning*, bot akan memutar radar secara terus-menerus agar area pemindaian menjadi lebih luas dan kemungkinan mendeteksi musuh menjadi lebih besar. Ketika musuh berhasil terdeteksi, bot akan menerima beberapa informasi penting, seperti posisi musuh, jarak terhadap musuh, arah pergerakan lawan, serta energi yang dimiliki musuh tersebut.

Data hasil *scanning* kemudian digunakan oleh bot untuk menentukan tindakan yang paling menguntungkan berdasarkan strategi greedy yang diterapkan. Oleh karena itu, proses *scanning* memiliki peran yang sangat penting karena kualitas keputusan bot sangat bergantung pada informasi yang berhasil diperoleh dari radar.

2.4 Evaluasi dan Pemilihan Aksi

Setelah memperoleh informasi dari proses *scanning*, bot akan melakukan evaluasi terhadap kondisi permainan untuk menentukan tindakan terbaik yang akan dilakukan. Evaluasi dilakukan dengan mempertimbangkan beberapa kondisi, seperti jarak musuh, posisi arena, energi bot, serta peluang serangan terhadap lawan.

Pendekatan greedy diterapkan dengan cara memilih aksi yang memberikan keuntungan paling besar pada saat itu. Bot akan menentukan prioritas tindakan berdasarkan *heuristic* yang telah dirancang sebelumnya. Tindakan yang dipilih dapat berupa menyerang musuh, menghindari serangan lawan, mengatur posisi, ataupun melakukan pergerakan tertentu untuk memperoleh keuntungan strategis.

Proses pemilihan aksi dilakukan secara cepat pada setiap *turn* permainan sehingga bot mampu merespons perubahan kondisi arena secara *real-time*. Keputusan yang telah dipilih tidak akan dievaluasi ulang, sesuai dengan karakteristik utama algoritma greedy (Jelita et al., 2025).

2.5 Eksekusi dan Perulangan Proses

Setelah tindakan terbaik dipilih, bot akan mengirimkan perintah aksi kepada *game engine* untuk dijalankan pada *turn* tersebut. Aksi yang dapat dilakukan bot meliputi bergerak maju atau mundur, memutar body tank, memutar radar, mengarahkan meriam, serta menembakkan peluru ke arah musuh.

Setelah aksi dijalankan, kondisi arena permainan akan berubah sesuai dengan interaksi yang terjadi antarbot. Bot kemudian akan kembali menerima informasi terbaru dari *game engine* dan mengulangi proses *scanning*, evaluasi, serta pengambilan keputusan pada *turn* berikutnya.

Proses ini berlangsung secara terus-menerus hingga ronde permainan selesai. Dengan adanya proses perulangan tersebut, bot dapat terus beradaptasi terhadap perubahan

kondisi arena dan mengambil keputusan secara dinamis selama permainan berlangsung.

2.6 Implementasi Strategi Greedy

Implementasi strategi greedy pada bot Robocode Tank Royale difokuskan pada proses pengambilan keputusan yang cepat dan efisien pada setiap *turn* permainan. Bot akan memilih tindakan yang dianggap paling menguntungkan berdasarkan kondisi arena saat itu tanpa melakukan pencarian seluruh kemungkinan solusi.

Strategi greedy diterapkan dalam berbagai aspek permainan, seperti pemilihan target musuh, penentuan arah pergerakan, pengaturan radar, serta penggunaan daya tembak. Setiap keputusan diambil berdasarkan *heuristic* tertentu yang dirancang untuk membantu bot memperoleh skor setinggi mungkin dalam permainan.

Pendekatan greedy dipilih karena sesuai dengan karakteristik permainan Robocode Tank Royale yang memiliki batas waktu eksekusi pada setiap *turn*. Dengan menggunakan strategi greedy, bot dapat mengambil keputusan secara cepat sehingga mampu merespons kondisi permainan secara efektif dan tetap kompetitif selama pertandingan berlangsung (Oktaviana & Naufal, 2017).

BAB III

APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Algoritma Greedy

Persoalan Robocode Tank Royale dapat dipetakan ke dalam elemen-elemen algoritma greedy sebagai berikut:

Tabel 1. *Mapping Elemen Algoritma Greedy pada Robocode Tank Royale.*

Elemen Greedy	Pemetaan dalam Robocode Tank Royale
Himpunan Kandidat	Semua aksi yang dapat dilakukan bot pada setiap turn: bergerak maju/mundur, berbelok, menembak dengan daya tembak 1/2/3, memindai musuh, atau tidak melakukan aksi tertentu.
Himpunan Solusi	Kombinasi aksi (gerakan + tembakan + pemindaian) yang dipilih bot pada setiap turn sehingga menghasilkan skor akhir pertempuran yang optimal.
Fungsi Seleksi	Fungsi yang memilih aksi terbaik berdasarkan kondisi saat ini, seperti jarak ke musuh, energi bot saat ini, dan status meriam (gun heat).
Fungsi Kelayakan	Suatu aksi layak jika: energi bot masih di atas nol, meriam tidak dalam keadaan terlalu panas (gun heat = 0), dan bot tidak sedang berada di luar batas arena.

Elemen Greedy	Pemetaan dalam Robocode Tank Royale
Fungsi Objektif	Memaksimalkan total skor akhir pertempuran yang merupakan akumulasi dari: Bullet Damage, Bullet Damage Bonus, Survival Score, Last Survival Bonus, Ram Damage, dan Ram Damage Bonus.

Pada setiap turn, bot menerapkan prinsip greedy dengan mengambil keputusan terbaik secara lokal berdasarkan informasi yang tersedia saat itu, yaitu posisi dan jarak musuh dari hasil pemindaian radar, serta kondisi energi dan status meriam bot sendiri. Keputusan yang diambil tidak mempertimbangkan konsekuensi jangka panjang, melainkan langsung memilih aksi yang memberikan manfaat maksimal pada turn tersebut.

3.2 Eksplorasi Alternatif Solusi Greedy

Berikut adalah empat alternatif solusi greedy yang dieksplorasi untuk persoalan Robocode Tank Royale. Setiap alternatif menggunakan heuristic yang berbeda sebagai dasar pengambilan keputusan.

3.2.1 Alternatif 1 — Distance-Based Adaptive Firepower (Bot Gacor)

Heuristic: Pilih daya tembak berdasarkan jarak ke musuh untuk memaksimalkan peluang peluru mengenai sasaran sekaligus memaksimalkan damage.

Strategi ini didasarkan pada mekanisme fisika peluru dalam Robocode: peluru dengan daya tembak tinggi (berat) bergerak lebih lambat sehingga lebih mudah meleset pada jarak jauh, sedangkan peluru dengan daya tembak rendah (ringan) bergerak cepat dan lebih andal pada jarak jauh meskipun damage-nya kecil. Bot secara greedy memilih firepower yang memberikan keuntungan optimal berdasarkan jarak saat musuh terdeteksi.

Tabel 2. Aturan Seleksi Firepower Bot Gacor.

Kondisi Jarak	Aksi Greedy yang Dipilih
Jarak < 150 piksel	Fire(3) — daya tembak maksimal, damage tinggi, cocok di jarak dekat
$150 \leq \text{Jarak} < 300$ piksel	Fire(2) — daya tembak sedang, keseimbangan antara kecepatan dan damage
Jarak ≥ 300 piksel	Fire(1) — daya tembak ringan, peluru cepat, lebih mudah mengenai target jauh

Gerakan utama bot adalah maju 50 unit lalu belok kanan 20 derajat secara berulang, membentuk pola melingkar. Radar berputar penuh 360 derajat setiap siklus untuk memastikan musuh selalu terpindai. Saat menabrak dinding, bot mundur 50 unit dan belok kanan 90 derajat.

3.2.2 Alternatif 2 — Energy-Aware Adaptive Strategy (Bot Syududu)

Heuristic: Sesuaikan strategi secara dinamis berdasarkan kondisi energi saat ini. Ketika energi tinggi, bot bermain agresif; ketika energi rendah, bot beralih ke mode defensif. Keputusan firepower juga tetap mempertimbangkan jarak seperti Alternatif 1, namun ditambah lapisan kontrol berbasis energi.

Strategi ini merupakan heuristic paling adaptif di antara keempat alternatif. Bot memiliki dua mode yang dipilih secara greedy berdasarkan ambang energi:

- Mode Agresif (Energi > 50): Bergerak maju 120 unit, tembak berdasarkan jarak (Fire 1/2/3), dan maju mengejar musuh saat terdeteksi.
- Mode Defensif (Energi ≤ 50): Mundur 80 unit dan belok 60° untuk menjauh dari musuh, serta mundur 100 unit saat musuh terdeteksi.

Tabel 3. Aturan Seleksi Firepower Bot Syududu.

Kondisi	Aksi Greedy yang Dipilih
Energi > 50, Jarak < 150	Fire(3) + Forward(50) — agresif di jarak dekat
Energi > 50, Jarak < 300	Fire(2) + Forward(50) — agresif di jarak sedang
Energi > 50, Jarak ≥ 300	Fire(1) + Forward(50) — agresif di jarak jauh
Energi ≤ 50 (apapun jaraknya)	Fire sesuai jarak + Back(100) — tembak lalu mundur
Kena peluru (OnHitByBullet)	TurnRight(120°) + Forward(100) — manuver menghindari
Gerakan default (Run, Energi > 50)	Forward(120) — bergerak aktif mencari musuh
Gerakan default (Run, Energi ≤ 50)	Back(80) + TurnRight(60°) — mundur dan menjauh

3.2.3 Alternatif 3 — Survival-First Evasion (Bot Kaciw)

Heuristic: Prioritaskan kelangsungan hidup (survival) dengan menghindari dari serangan secara reaktif. Setiap kali kena peluru, langsung ambil aksi menghindari terbaik saat itu, dengan mengorbankan potensi damage untuk mempertahankan Survival Score.

Strategi ini memanfaatkan komponen skor Survival Score (50 poin setiap bot lain mati) dan Last Survival Bonus. Bot tidak agresif dalam mengejar damage, melainkan fokus bertahan selama mungkin. Gerakan bot bersifat acak (random) untuk membuat lintasan bot tidak dapat diprediksi musuh, sehingga lebih sulit untuk ditembak. Saat kena peluru, bot langsung berbelok dan lari menjauh secara greedy.

Tabel 4. Aturan Seleksi Firepower Bot Kaciw.

Event / Kondisi	Aksi Greedy yang Dipilih
Bot musuh terpindai (OnScannedBot)	Fire(1) saja — tembak minimal untuk hemat energi
Kena peluru (OnHitByBullet)	TurnRight(90–180° random) + Forward(120) — kabur segera
Gerakan default (Run loop)	Maju 100 unit + belok kanan random (30–90°) + scan 360°
Menabrak dinding (OnHitWall)	Mundur 80 unit + belok kanan random (45–135°)

3.2.4 Alternatif 4 — Always-Aggressive Maximum Damage (Bot Kacau)

Heuristic: Selalu tembak dengan daya tembak maksimal (Fire(3)) tanpa mempertimbangkan kondisi apapun, dengan keyakinan bahwa serangan konstan dan agresif akan menghasilkan total damage tertinggi.

Strategi ini menerapkan prinsip greedy paling sederhana: setiap kali ada kesempatan menembak, selalu gunakan daya tembak tertinggi. Bot tidak membuang waktu untuk kalkulasi jarak atau kondisi energi. Selain itu, saat bertabrakan dengan bot lain (HitBotEvent), bot justru maju terus sambil menembak untuk memaksimalkan Ram Damage sekaligus Bullet Damage secara bersamaan.

Tabel 5. Aturan Aksi Greedy Bot Kacau.

Event / Kondisi	Aksi Greedy yang Dipilih
Bot musuh terpindai (OnScannedBot)	Fire(3) langsung + maju 80 unit + putar meriam 20°
Bertabrakan dengan bot lain (OnHitBot)	Fire(3) + maju 50 unit (memaksimalkan ram damage)

Event / Kondisi	Aksi Greedy yang Dipilih
Gerakan default (Run loop)	Maju 150 unit, belok kanan 25°, scan 360°, Fire(2)
Menabrak dinding (OnHitWall)	Mundur 100 unit + belok kanan 90°

3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi

Berikut adalah analisis efisiensi (kompleksitas komputasi per *turn*) dan efektivitas (kemampuan memaksimalkan skor) dari keempat alternatif solusi greedy:

Tabel 6. Perbandingan Efisiensi dan Efektivitas Keempat Alternatif Solusi.

Aspek	Gacor (Alt. 1)	Syududu (Alt. 2)	Kaciw (Alt. 3)	Kacau (Alt. 4)
Kompleksitas per turn	O(1)	O(1)	O(1)	O(1)
Kondisi yang dipertimbangkan	Jarak musuh saja	Jarak + Energi bot	Reaksi terhadap event bahaya	Tidak ada (selalu agresif)
Efektivitas Bullet Damage	Tinggi	Tinggi + adaptif	Rendah (Fire(1) saja)	Sangat Tinggi (tetapi boros energi)
Efektivitas Survival	Sedang	Tinggi (mode defensif)	Tinggi	Rendah (mudah mati)
Efektivitas Ram Damage	Tidak ada	Tidak ada	Tidak ada	Tinggi
Kelemahan utama	Tidak ada respons saat energi rendah	Lebih kompleks, banyak kondisi edge case	Damage sangat kecil	Boros energi, mudah mati duluan

Aspek	Gacor (Alt. 1)	Syududu (Alt. 2)	Kaciw (Alt. 3)	Kacau (Alt. 4)
Kekuatan utama	Efisien dan efektif di semua jarak	Paling adaptif dan seimbang	Survival paling lama	Damage maksimal dan Ram Damage

Semua alternatif memiliki kompleksitas **O(1)** per *turn* karena keputusan diambil berdasarkan perbandingan nilai konstan (jarak, energi) tanpa iterasi atau struktur data yang bergantung pada jumlah musuh. Hal ini sangat penting mengingat adanya batas waktu turn (turn timeout) sebesar **30–50 ms** yang harus dipatuhi setiap bot.

3.4 Strategi Greedy yang Dipilih dan Alasan Pemilihan

Berdasarkan analisis pada subbab 3.3, strategi greedy yang dipilih sebagai bot utama adalah **Alternatif 1: Distance-Based Adaptive Firepower (Bot Gacor)**.

Alasan Pemilihan:

- **Fungsi Objektif Langsung pada Bullet Damage:** Gacor mengoptimalkan komponen skor terbesar secara langsung, yaitu Bullet Damage, dengan memilih firepower yang paling proporsional terhadap jarak. Fire(3) di jarak dekat memaksimalkan damage sekaligus peluang kena, sementara Fire(1) di jarak jauh memastikan peluru ringan yang cepat tetap bisa mengenai musuh.
- **Heuristic Sederhana dan Deterministik:** Keputusan greedy berbasis satu variabel (jarak) membuat bot sangat mudah diprediksi perilakunya saat demo, dan mudah dijelaskan kesesuaiannya antara laporan dengan implementasi. Ini penting karena 20% nilai berasal dari kesesuaian strategi yang ditulis dengan yang diimplementasikan.
- **Risiko Rendah terhadap Kesalahan Logika:** Dibandingkan strategi dengan banyak kondisi bercabang, Gacor memiliki alur keputusan yang sangat bersih: satu fungsi **DistanceTo**, satu blok **if-else** di **OnScannedBot**. Tidak ada state yang perlu dijaga antar turn sehingga tidak ada risiko bug dari kondisi yang tidak terduga.
- **Gerakan Konsisten Mendukung Pemindaian:** Pola gerak melingkar (maju 50 + belok kanan 20°) dikombinasikan dengan **TurnGunRight(360)** memastikan radar selalu menyapu arena setiap siklus, sehingga musuh hampir selalu terpindai dan keputusan greedy selalu punya data untuk diproses.

Bot Kacau (Alternatif 4) tidak dipilih karena strategi agresif tanpa kendali energi sangat berisiko kehabisan energi lebih cepat dari musuh. **Bot Kaciw** (Alternatif 3) tidak dipilih karena total damage yang dihasilkan terlalu kecil dengan Fire(1) saja untuk bersaing dalam akumulasi skor akhir. **Bot Syududu** (Alternatif 2) tidak dipilih sebagai bot utama karena kompleksitas logikanya lebih tinggi dibandingkan Gacor, sementara keunggulan adaptifnya belum tentu signifikan dalam pertempuran singkat.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Pseudocode Keempat Alternatif Solusi

Bagian ini berisi algoritma dari 4 alternatif bot yang Anda rancang dalam bentuk *pseudocode* yang detail agar mudah dipahami.

4.1.1 Pseudocode Bot Gacor (Bot Utama)

```
PROGRAM Gacor
```

```
DEKLARASI:
```

```
    enemyX, enemyY : real // koordinat musuh terakhir
```

```

PROSEDUR Run():
    SET warna body, turret, radar
    SELAMA bot masih berjalan:
        TurnGunRight(360) // sweep radar penuh setiap siklus
        Forward(50)       // maju 50 unit
        TurnRight(20)      // belok kanan 20 derajat
                          // membentuk pola gerak melingkar

PROSEDUR OnScannedBot(e):
    // Greedy: pilih firepower berdasarkan jarak saat ini
    jarak <- DistanceTo(e.X, e.Y)
    JIKA jarak < 150 MAKA
        Fire(3)           // jarak dekat: peluru berat, damage
maksimal
    SELAIN JIKA jarak < 300 MAKA
        Fire(2)           // jarak sedang: keseimbangan damage &
kecepatan
    SELAIN
        Fire(1)           // jarak jauh: peluru ringan, cepat, mudah
kena
    AKHIR JIKA

PROSEDUR OnHitWall(e):
    Back(50)              // mundur dari dinding
    TurnRight(90)         // belok 90 derajat

FUNGSI DistanceTo(x, y) -> real:
    dx <- X - x
    dy <- Y - y
    KEMBALIKAN sqrt(dx*dx + dy*dy)

AKHIR PROGRAM

```

4.1.2 Pseudocode Bot Syududu (Alternatif 1)

```

PROGRAM Syududu

DEKLARASI:
    enemyX, enemyY : real // posisi musuh terakhir
    enemyDistance  : real // jarak ke musuh

PROSEDUR Run():
    SET warna body, turret, radar
    SELAMA bot masih berjalan:
        // Greedy: pilih mode berdasarkan energi saat ini
        JIKA Energy > 50 MAKA
            Forward(120) // energi aman -> mode agresif

```

```

        SELAIN
            Back(80)          // energi rendah -> mode defensif
            TurnRight(60)     // mundur dan menjauh
        AKHIR JIKA
        TurnGunRight(360)    // sweep radar

PROSEDUR OnScannedBot(e):
    enemyDistance <- DistanceTo(e.X, e.Y)

    // Greedy lapis 1: pilih firepower berdasarkan jarak
    JIKA enemyDistance < 150 MAKA
        Fire(3)
    SELAIN JIKA enemyDistance < 300 MAKA
        Fire(2)
    SELAIN
        Fire(1)
    AKHIR JIKA

    // Greedy lapis 2: pilih gerakan berdasarkan energi
    JIKA Energy < 30 MAKA
        Back(100)           // energi kritis -> mundur jauh
    SELAIN JIKA Energy > 50 MAKA
        Forward(50)         // energi aman -> maju mengejar
    // Jika 30 <= Energy <= 50: diam, fokus tembak
    AKHIR JIKA

PROSEDUR OnHitByBullet(e):
    TurnRight(120)
    Forward(100)

PROSEDUR OnHitWall(e):
    Back(80)
    TurnRight(90)

FUNGSI DistanceTo(x, y) -> real:
    KEMBALIKAN sqrt((X-x)^2 + (Y-y)^2)

AKHIR PROGRAM

```

4.1.3 Pseudocode Bot Kaciw (Alternatif 2)

```

PROGRAM Kaciw

DEKLARASI:
    random : objek Random // generator angka acak untuk gerakan
    unpredictable

PROSEDUR Run():

```



```

    SET warna body, turret, radar
    SELAMA bot masih berjalan:
        Forward(100)                // maju dengan jarak tetap
        TurnRight(random(30, 90))   // belok acak agar
Lintasan tidak terprediksi
        TurnGunRight(360)           // sweep radar penuh

PROSEDUR OnScannedBot(e):
    // Greedy: tembak minimal untuk hemat energi, prioritas survival
    Fire(1)                         // firepower minimum

PROSEDUR OnHitByBullet(e):
    // Greedy: kena peluru -> langsung kabur ke arah acak
    TurnRight(random(90, 180))      // belok acak menjauhi
arah peluru
    Forward(120)                    // lari jauh

PROSEDUR OnHitWall(e):
    Back(80)
    TurnRight(random(45, 135))      // belok acak menjauhi
dinding

AKHIR PROGRAM

```

4.1.4 Pseudocode Bot Kacau (Alternatif 3)

```

PROGRAM Kacau

PROSEDUR Run():
    SET warna body, turret, radar
    SELAMA bot masih berjalan:
        Forward(150)                // maju agresif
        TurnRight(25)               // belok kanan
        TurnGunRight(360)           // sweep radar
        Fire(2)                     // tembak default di loop utama

PROSEDUR OnScannedBot(e):
    // Greedy: selalu tembak maksimal tanpa pertimbangan kondisi
    Fire(3)                         // daya tembak selalu maksimal
    Forward(80)                     // terus mendekat ke musuh
    TurnGunRight(20)                // putar gun sedikit untuk tracking

PROSEDUR OnHitBot(e):
    // Greedy: manfaatkan tabrakan untuk ram damage
    Fire(3)                         // tembak saat menabrak
    Forward(50)                     // terus maju untuk tambah ram damage

PROSEDUR OnHitWall(e):

```

```
Back(100)
TurnRight(90)

AKHIR PROGRAM
```

4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur Bot Utama (Gacor)

Berikut adalah penjelasan lengkap struktur data, fungsi, dan prosedur yang digunakan pada Bot Gacor sebagai bot utama yang dipilih.

4.2.1 Struktur Data

Tabel 7. Struktur Data Bot Gacor.

Nama	Tipe	Keterangan
X, Y	double (bawaan API)	Koordinat posisi bot saat ini di arena, diperbarui otomatis oleh game engine setiap turn.
GunDirection	double (bawaan API)	Arah gun saat ini dalam derajat (0° = atas, searah jarum jam). Digunakan sebagai referensi saat memutar gun.
Energy	double (bawaan API)	Energi bot saat ini. Dimulai dari 100, berkurang saat ditembak atau menembak, bertambah saat peluru mengenai musuh.
IsRunning	bool (bawaan API)	Flag dari game engine yang bernilai true selama bot masih aktif dalam

Nama	Tipe	Keterangan
		pertempuran. Digunakan sebagai kondisi loop utama di Run().

4.2.2 Fungsi

Tabel 8. *Daftar Fungsi Bot Gacor.*

Nama Fungsi	Penjelasan
DistanceTo(x, y)	Menghitung jarak Euclidean dari posisi bot saat ini (X, Y) ke titik (x, y) menggunakan rumus $\sqrt{(X-x)^2 + (Y-y)^2}$. Mengembalikan nilai bertipe double. Digunakan di OnScannedBot untuk menentukan jarak ke musuh sebagai input keputusan greedy.

4.2.3 Prosedur

Tabel 9. *Daftar Prosedur Bot Gacor.*

Nama Prosedur	Penjelasan
Run()	Prosedur utama yang berjalan dalam loop selama bot aktif. Setiap iterasi: memutar gun 360° (sweep radar), maju 50 unit, lalu belok kanan 20°. Pola ini membentuk gerakan melingkar yang memastikan musuh selalu terpindai setiap beberapa turn.

OnScannedBot (e)	Event handler yang dipanggil otomatis oleh game engine setiap kali radar mendeteksi musuh. Berisi inti logika greedy: menghitung jarak ke musuh menggunakan DistanceTo(), lalu memilih firepower (1, 2, atau 3) secara greedy berdasarkan tiga threshold jarak.
OnHitWall(e)	Event handler yang dipanggil saat bot menabrak dinding arena. Bot mundur 50 unit lalu belok kanan 90° untuk menghindari wall damage yang terus-menerus.

4.2.4 Alur Keputusan Greedy

Alur pengambilan keputusan greedy Bot Gacor pada setiap turn dapat dijelaskan sebagai berikut:

- Turn dimulai: game engine memanggil Run() secara berulang.
- Gun berputar 360° untuk memindai seluruh arena.
- Jika radar mendeteksi musuh, game engine memanggil OnScannedBot(e).
- DistanceTo() menghitung jarak ke musuh secara real-time.
- Keputusan greedy diambil: jarak < 150 → Fire(3), jarak < 300 → Fire(2), selain itu → Fire(1).
- Bot maju 50 unit dan belok 20° untuk melanjutkan pola melingkar.
- Jika menabrak dinding, OnHitWall() dipanggil dan bot menghindar.

4.3 Pengujian (Hasil Pertandingan)

Pengujian wajib dilakukan minimal **3 kali** dalam format pertandingan **1 vs 1** melawan bot asisten/standar. Hasilnya dalam bentuk tabel berikut ini:

Tabel 10. Hasil Pengujian dengan bot asisten.

No	Nama Lawan	Hasil Laga	Skor Bot Utama	Skor Bot Lawan	Kejadian Unik/ Kondisi Lapangan
1	Fire 1.0	Menang	1580	116	Fire 1.0 diam di

No	Nama Lawan	Hasil Laga	Skor Bot Utama	Skor Bot Lawan	Kejadian Unik/ Kondisi Lapangan
					suatu tempat lalu <i>scanning</i> musuh, tetapi jika dia terkena tembakan dari bot utama maka dia akan bergeser.
2	Crazy 1.0	Menang	316	173	Crazy 1.0 tidak melakukan tembakan sama sekali dan bergerak secara acak, sehingga bot utama sedikit kesusahan untuk menargetkan lawan.
3	Corners 1.0	Menang	1357	350	Corners 1.0 berada di posisi pojok untuk <i>scanning</i> bot utama, lalu melakukan tembakan yang nyaris selalu mengenai bot utama.

4.4 Analisis Hasil Pengujian

Berdasarkan data dari tiga pertandingan uji coba 1 vs 1 melawan bot asisten standar, berikut adalah analisis mendalam mengenai performa, efektivitas, serta kondisi keterbatasan dari strategi *Greedy* yang diterapkan pada Bot Utama:

4.4.1 Analisis Performa Kemenangan Berdasarkan Komponen Skor

Secara keseluruhan, Bot Utama menunjukkan performa yang sangat dominan dengan memenangkan seluruh pertandingan. Keberhasilan ini dipengaruhi oleh optimalisasi fungsi objektif *greedy* terhadap komponen skor permainan:

- **Kasus Lawan Fire 1.0 (Skor: 1580 vs 116):** Bot Utama mencapai efisiensi tertinggi pada laga ini. Sifat Fire 1.0 yang cenderung diam di satu tempat mempermudah fungsi seleksi *greedy* berbasis jarak (*Greedy by Distance*) untuk mengunci koordinat target secara konstan. Karena jarak yang sangat dekat, deviasi peluru menjadi minimal, menghasilkan *Bullet Damage* maksimal secara berturut-turut. Pengembalian energi sebesar tiga kali lipat dari setiap peluru yang masuk memastikan Bot Utama mempertahankan stamina energinya hingga akhir laga sembari mengamankan *Bullet Damage Bonus* saat mengeliminasi lawan.
- **Kasus Lawan Crazy 1.0 (Skor: 316 vs 173):** Pertandingan ini menghasilkan akumulasi skor yang jauh lebih rendah bagi Bot Utama. Kondisi lapangan menunjukkan pergerakan Crazy 1.0 yang sangat acak membuat arah sapuan radar (*scan arc*) Bot Utama sering kali terlambat mendeteksi koordinat terbaru. Akibatnya, beberapa keputusan menembak instan (*local optimum*) meleset dari target. Kendati demikian, keunggulan strategi *greedy* terletak pada persistensinya; begitu jarak berhasil dipangkas, Bot Utama tetap mampu mendaratkan kerusakan krusial dan menang lewat akumulasi *Survival Score*.
- **Kasus Lawan Corners 1.0 (Skor: 1357 vs 350):** Corners 1.0 memberikan perlawanan paling ketat dengan mencetak skor 350. Karakteristik lawan yang mengunci posisi di sudut arena memungkinkannya meluncurkan tembakan yang hampir selalu mengenai badan Bot Utama. Namun, keunggulan skor telak 1357 diraih karena Bot Utama secara agresif terus bergerak mendekati sudut tersebut. Berada di jarak yang sangat rapat membuat daya tembak peluru Bot Utama

meningkat ke tingkat maksimum (daya berat), mengompensasi kerusakan yang diterima dengan *damage* balasan yang jauh lebih destruktif.

4.4.2 Kapan Strategi Greedy Berhasil Mencapai Nilai Optimal?

Strategi *Greedy by Distance* yang dikombinasikan dengan daya tembak dinamis ini **terbukti mencapai nilai optimal** ketika berhadapan dengan musuh yang memiliki pola pergerakan statis, semi-statis, atau terprediksi (seperti Fire 1.0 dan Corners 1.0).

Secara teoritis, dengan memprioritaskan elemen kandidat berjarak minimum, bot memperkecil waktu layang peluru di udara. Probabilitas peluru mengenai sasaran mendekati 100%, sehingga siklus ekonomi energi tank (menembak - mengenai musuh - mendapat energi 3 kali) berjalan sempurna. Hal ini mengoptimalkan komponen skor *Bullet Damage* dan *Bullet Damage Bonus* secara masif.

4.4.3 Kapan Strategi Greedy Gagal Mencapai Nilai Optimal?

Meskipun menyapu bersih kemenangan, pengujian menunjukkan bahwa strategi ini **gagal mencapai nilai optimal atau mengalami degradasi efisiensi** dalam kondisi lapangan sebagai berikut:

- **Menghadapi Musuh dengan Agilitas Tinggi dan Pola Acak (*High Mobility*):** Seperti pada kasus Crazy 1.0, pergerakan acak mengeksploitasi kelemahan mendasar *greedy* yang tidak melakukan kalkulasi prediksi posisi masa depan (*predictive targeting*). Bot hanya menembak ke posisi musuh saat radar mendeteksinya di *turn* aktif, sehingga peluru sering berakhir di ruang kosong karena musuh sudah berpindah tempat.
- **Kebutaan Terhadap Geometri Arena (Sudut Dinding):** Saat menghadapi Corners 1.0, hasrat instan untuk mengunci target terdekat di pojok arena berpotensi memicu bahaya laten berupa *wall damage*. Karena fungsi seleksi hanya memikirkan parameter jarak target tanpa mengevaluasi fungsi kelayakan koordinat posisi sendiri terhadap batas dinding, tank rentan menabrak batas arena secara berulang yang dapat mengurangi energi secara cuma-cuma.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian bot Robocode Tank Royale menggunakan algoritma greedy, dapat disimpulkan bahwa:

- Algoritma greedy berhasil diterapkan pada bot dengan kompleksitas $O(1)$ per turn, sesuai dengan batas waktu turn timeout 30–50 ms yang ditetapkan oleh game engine.
- Dari keempat alternatif yang dirancang, Bot Gacor (Distance-Based Adaptive Firepower) dipilih sebagai bot utama karena secara langsung mengoptimalkan komponen skor terbesar (Bullet Damage) melalui pemilihan firepower adaptif berdasarkan jarak musuh.
- Pengujian menunjukkan Bot Gacor memenangkan seluruh pertandingan uji 1 vs 1 dengan skor 1.580 vs 116 (Fire 1.0), 316 vs 173 (Crazy 1.0), dan 1.357 vs 350 (Corners 1.0), membuktikan efektivitas strategi greedy berbasis jarak.

- Strategi greedy optimal saat menghadapi musuh berpola statis atau terprediksi, namun kurang efektif terhadap musuh bermobilitas tinggi dan bergerak acak karena tidak memperhitungkan prediksi posisi musuh di masa depan.

5.2 Saran

Beberapa saran untuk pengembangan bot ke depannya adalah sebagai berikut:

- Menambahkan predictive targeting agar peluru lebih akurat mengenai musuh yang bergerak acak.
- Mengintegrasikan mekanisme energy-aware sehingga bot dapat beralih ke mode defensif saat energi kritis.
- Menambahkan deteksi jarak terhadap batas arena untuk meminimalkan wall damage yang tidak perlu.

LAMPIRAN

Lampiran A — Tautan Repository GitHub

Repository tugas besar ini dapat diakses pada tautan berikut:

[wulanlubis/Tubes_KacauMania](https://github.com/wulanlubis/Tubes_KacauMania)

Struktur repository:

- src/main-bot/ — source code Bot Gacor (bot utama)
- src/alternative-bots/Kacau/ — source code Bot Kacau
- src/alternative-bots/Kaciw/ — source code Bot Kaciw
- src/alternative-bots/Syududu/ — source code Bot Syududu
- doc/KacauMania.pdf — laporan tugas besar dalam format PDF
- README.md — panduan instalasi dan penggunaan

Lampiran B — Tautan Video YouTube

<https://youtu.be/dYMyWuAcMOE>

Lampiran C — Tabel Checklist Spesifikasi

Tabel 10. Hasil Pengujian dengan bot asisten.

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten.		
2	Membuat 4 solusi greedy dengan heuristic yang berbeda.		
3	Membuat laporan sesuai dengan spesifikasi.		
4	Membuat video bonus dan diunggah pada Youtube.		

DAFTAR PUSTAKA

- HARIANJA, A. P. (2019, 01 01). *PERANCANGAN PERANGKAT LUNAK VISUALISASI ALGORITMA GREEDY*, 3(1), 27–39.
- Jelita, F., Fallo, D., & Miru, Y. G. (2025, 06 28). Optimalisasi Rute Menggunakan Algoritma Dijkstra dan Greedy: Sebuah Pendekatan Komparatif. *Jurnal Kridatama Sains Dan Teknologi*, 07(01), 555–562.
- Munir, R. (n.d.). *Modul Algoritma Greedy Bagian 1*. STEI-ITB.
- Oktaviana, S., & Naufal, A. (2017, 05). Algoritma Greedy untuk Optimalisasi Ruangan dalam Penyusunan Jadwal Perkuliahan. *JURNAL MULTIMEDIA NETWORKING INFORMATICS*, 3(1), 54–59.

